SC

SOCIAL CONSOLE

Security & Account Protection Guide

Understanding how your account, data, authentication, and publishing activities are protected.

This guide describes the protections implemented in the Social Console admin application and its publishing worker. Sections marked 'Enforced by the application' describe server-side behavior. Section 7 lists recommended administrator practices, which are marked separately.

Contents

1. Account Security 3
2. Authentication & Password Protection 4
3. Session Management 5
4. Social Account & OAuth Security 6
5. Publishing Safety 7
6. Data Protection & Privacy 8
7. Administrative Responsibilities 9

About this guide

This document is generated from the Social Console project and describes behavior that is implemented and enforced in the admin web application and its publishing worker. It contains no credentials, tokens, or private account data.

SECTION 1

Account Security

Social Console is administered by a single administrator account: the Super Admin. The protections in this section are built into the application and its database schema, so they hold without any optional configuration.

What is enforced

- Only one Super Admin account can exist. A database-level unique index on the administrator role prevents a second admin account from ever being created.
- New registrations do not receive access automatically. Every new account starts in a pending state and must be approved by the Super Admin before it can sign in.
- Suspended and deleted accounts cannot use the application. When an account is suspended or soft-deleted, its sessions are revoked and its scheduled posts are held so nothing is published on its behalf.
- The Super Admin account cannot be suspended, deleted, or downgraded. Safeguards protect the only account able to administer the console.
- Admin privileges are verified server-side on every request. The role is never trusted from the browser: admin pages and admin APIs re-check the live session and the account state before doing any work.

> **Enforced by the application**
> Everything in this section is enforced by the server and the database. No administrator action is required to keep these protections active.

Administrative changes to individual user accounts are made from the Users pages, not from Settings.

SECTION 2

Authentication & Password Protection

Authentication is provided by Better Auth. Passwords are stored only in hashed form and are verified by the authentication library at sign-in. Plain-text passwords are never persisted, displayed, or returned by an API.

Password requirements

- A password must be at least 12 characters long (up to 128).
- It must contain characters from at least three of these four classes: lowercase letters, uppercase letters, numbers, and symbols.
- The rules are checked instantly in the settings form and are validated again on the server before any password change is accepted.

Changing your password

- Changing the password requires your current password, so a stolen unlocked browser alone is not enough.
- Better Auth performs the update and revokes every other signed-in session in the same operation, so a password change immediately signs out unknown devices.
- The change is recorded in the activity infrastructure as an administrative security event.

Email one-time codes

- Where email verification is configured, the console uses 6-digit one-time codes that expire after 5 minutes, allow at most 3 attempts, and enforce a 60-second cooldown between resend requests.
- One-time codes are stored in hashed form and are never written to logs or audit metadata.
- Codes are delivered by email through the configured transport (Resend or a generic HTTPS delivery webhook). Delivery failures are deliberately indistinguishable from an unknown email address, so responses do not reveal whether an account exists.
- Completing a password reset through a one-time code revokes every existing session once the new password is set.

> **Enforced by the application**
> Password rules, hashed credential storage, one-time-code limits, and session revocation on password changes and resets are all enforced server-side.

SECTION 3

Session Management

Every sign-in creates an independent server-side session. The browser only holds a signed session cookie; the session record itself lives in the database and can be inspected and revoked from Settings.

- Sessions expire after 7 days, and end earlier when you sign out, change your password, or complete a password reset.
- The Active sessions list in Settings shows every browser currently signed in, with the device and browser, the last active time, and the expiration time.
- Your current device is marked as the current session so you can tell it apart from other devices at a glance.
- The Sign out other sessions action revokes every session except the one you are using, after a confirmation step.
- Every request re-validates the session server-side. A revoked or expired session is rejected immediately, even if the browser still holds the cookie.

> **Enforced by the application**
> Session creation, expiration, server-side validation, and revocation are handled by the authentication library and verified on every admin request. Session tokens are never displayed in the dashboard.

SECTION 4

Social Account & OAuth Security

Publishing works through social accounts that you connect from Facebook, Instagram, and LinkedIn. Connections are authorized with OAuth: you approve access on the provider's site, and the console never sees your social network password.

- OAuth access tokens and refresh tokens are encrypted with AES-256-GCM before they are stored. The encryption key is held in server-side environment configuration and is never written to the database.
- Tokens and app credentials are never displayed in the dashboard and are never returned by an API response.
- Only non-secret connection metadata, such as the account or page name, the platform, and the connection status, is shown in the console.
- Expired or invalid connections fail safely. A publish attempt against a bad connection fails and is recorded instead of silently continuing; the account can then be reconnected.

Enforced by the application
Encryption of social tokens at rest and the omission of secrets from every dashboard and API response are implemented in the publishing worker and its credential store.

SECTION 5

Publishing Safety

Scheduling is backed by PostgreSQL. The publishing worker claims due posts with row-level locks so that a post is processed by exactly one worker, exactly once.

- Due posts are claimed with SELECT ... FOR UPDATE SKIP LOCKED. The row lock makes duplicate publishing impossible, even if several workers run at the same time.
- The claim query refuses posts that belong to suspended or deleted accounts. This hold is enforced at the database level, so a suspension and a worker claim cannot race each other.
- Held posts stay held until an administrator resumes them. Resuming an overdue held post is blocked; the post must be rescheduled instead of being published immediately, which prevents surprise duplicate or late publishing.
- Every create, hold, resume, retry, publish, and failure is recorded in the activity infrastructure with a scheduled_post action.

> **Enforced by the application**
> Row-level claim locking, account-state enforcement, hold behavior, and audit logging of publishing actions are part of the scheduler's core transaction logic.

Administrators can review every scheduled post, including held and failed ones, from the Scheduler page.

SECTION 6

Data Protection & Privacy

- Sensitive values such as social tokens, OAuth secrets, and passwords are never displayed in the dashboard and are never returned by an admin API.
- Social account tokens remain encrypted at rest with AES-256-GCM.
- Activity metadata is sanitized before it is stored. Passwords, tokens, secrets, and encryption keys never enter activity records.
- Every admin API request verifies the Super Admin session server-side before it processes data.
- Database access runs server-side through an authorized service connection. An optional hardening migration additionally enables row-level security on the scheduling and authentication tables and revokes direct browser-facing access to them.
- Deleting a user is a soft delete that preserves audit history, and account relationships are protected by database foreign keys.

> **Enforced by the application**
> Sanitization, encryption, server-side authorization, and audit-preserving deletion are implemented in the application code and reviewed by the automated test suite.

SECTION 7

Administrative Responsibilities

Most protections described in this guide run automatically. A smaller set of good practices depends on you as the administrator. These are recommendations, not enforced features.

Recommended practices

- Use a strong, unique administrator password that meets the console minimum of 12 characters with three of four character types, and do not reuse it on other services.
- Never share administrator credentials or one-time codes with anyone. No legitimate support process will ask for them.
- Sign out other sessions from Settings when a device is lost, replaced, or no longer used.
- Review social account connection statuses periodically and reconnect any expired or invalid connection before its next scheduled post.
- Review scheduled posts before their publish window and hold anything that is not ready to go out.
- Keep the authentication and email delivery configuration valid and up to date so one-time codes and security messages actually arrive.

> **Recommended practice - not enforced**
> The items in this section are operational advice for administrators. They complement, but are not enforced by, the application.